

# Proposed layer\_type Extension: formation

*Documented rationale for a GLM controlled-vocabulary addition (schema\_version 1.0 → 1.1).*

Status: submitted for formal review.

Proposing implementation: NOUS.

Standard maintainer reviewing: EVIDE Governance Lab.

Prepared under the GLM vocabulary extension mechanism: documented rationale plus schema version increment.

---

## 1. Purpose and submission path

### 1.1 What is proposed.

This document proposes the addition of one value, `formation`, to the controlled `layer_type` vocabulary of the Governance Layer Manifest (GLM), under the extension mechanism the standard defines: a documented rationale plus a schema version increment. This document is the documented rationale. The requested increment is non-breaking, `schema_version` 1.0 to 1.1, per Section 10. The proposed entry (timing surface, operational function, and definition) is given in Section 5; Sections 2 through 4 establish the category it names; Sections 6 through 8 declare its claims boundary and composition posture; Section 9 documents the implementation that motivates it.

### 1.2 What is not proposed.

No existing type is modified. No field is added. No new timing position is introduced. No authority, certification, or privileged status is claimed for the proposing implementation or the proposed type; a GLM manifest is “self-declared infrastructure metadata”, and this proposal inherits that posture in full. A conforming `formation` manifest is not attached: it follows upon acceptance of the category, per the sequencing established in the proposing parties’ prior written exchange — category first, manifest after.

### 1.3 Origin.

The proposal originates in a written exchange between the proposing parties that ran the accommodation question first: whether the existing `witness` type can absorb a `formation` proof object without redefinition. The exchange reached, in writing, the conclusion that it cannot without producing the semantic collapse the standard is designed to prevent. This document formalizes that conclusion through the standard’s own method and vocabulary.

## 2. Definitions

### 2.1 Formation proof object.

A sealed, independently verifiable artifact establishing that declared conditions must hold across every execution path derivable from a specific source artifact, formed before any execution exists. It establishes that conditions must hold; it does not record that conditions held during any specific execution. This definition is stated once and carried verbatim into the manifest definitions entry of Section 5; the document and a conforming manifest must not carry divergent definitions of the same term.

### 2.2 Universal quantification over execution paths.

The proof ranges over the complete set of execution paths derivable from the source artifact, not over any executed, observed, or sampled path. Its subject is the set; no member of the set needs to exist for the proof to hold. This is the operational meaning of “must hold”: the claim is about every possible execution, established before any actual execution.

### **2.3 Source-anchored timing.**

The temporal reference frame of a formation proof is the source artifact, not a bind event. “Pre-bind” therefore means prior to and independent of any execution instantiation, not earlier on the timeline of one particular bind. A formation proof formed once holds unchanged for every subsequent bind of the same source; a change to the source requires a new proof.

### **2.4 Non-claim and forbidden inference.**

A non-claim declares what the layer does not do. A forbidden inference declares what a consumer of the layer’s outputs must not conclude. The second is the stronger guarantee: it constrains the reader, not the author. This document states the proposed type’s non-claims in `operational_scope` and `claims_boundary`, and its forbidden inferences in `forbidden_interpretations`, following the GLM v2.0 field semantics.

## **3. The mapping test**

### **3.0 Method.**

Each existing functional type is tested for whether its definition can accommodate a formation proof object without stretching. The test follows the sequence agreed in the proposing parties’ prior exchange: accommodation first, a new category only if accommodation fails. `cross_cutting` is excluded from the test: as the table structure indicates, its operational-function entry specifies how timing surfaces must be declared rather than naming a governance operation, making it a positional declaration rather than a functional category. Formation occupies a single declared surface.

### **3.1 substrate.**

Defined as: “Records behavioral history and surfaces it as an independently queryable signal at decision time.” The operation is accumulation: observations of behavior that occurred. A formation proof has no behavioral input; its object exists before any behavior does. The shared pre-bind surface is exactly where the parentheses separate the two: substrate is continuous accumulation along the timeline, formation is a single act anchored to the source artifact.

### **3.2 boundary.**

Defined as: “Resolves admissibility at execution bind-time before consequence commits.” The operation is a decision, taken at bind, with execution consequence. Formation decides nothing and has no operational presence at bind. Its output is an input: a boundary layer may consume a formation proof as one admissibility condition, which is composition between the types, not identity.

### **3.3 closure.**

Defined as: “Stabilizes responsibility-bearing closure state as an externally verifiable artifact that does not require access to the originating system for interpretation.” The operation is responsibility crystallization after bind. Formation and closure are mirror positions on the timing axis, formed before drift on one side and before execution on the other, and the mirror relation presupposes the distinction: opposite ends of an

axis are not one category. The objects differ accordingly: a formation proof on one side, an accountable configuration on the other.

### **3.4 reviewability.**

Defined as: “Tests whether artifacts produced by other layers satisfy conditions for independent review without access to the originating environment.” The operation is second-order: a property test over other layers’ outputs. A formation proof object is first-order, an artifact such a test would take as input. That a formation dossier is built to pass that test makes it a well-behaved subject of reviewability, not an instance of it.

### **3.5 witness.**

Defined as: “Preserves independently checkable evidence of governed-state conditions surrounding the boundary.” This is the closest type and the one the accommodation test must take seriously.

The timing reading nests: before-or-at-bind includes before-bind, so on the clock alone a pre-execution proof fits inside the witness window. Timing therefore cannot carry the separation, and this document does not rest it there.

The separation is the epistemic operation. A witness records that conditions held: an existential claim about a specific boundary context, whose evidentiary value derives from the observation having occurred. A formation proof establishes that conditions must hold: a universal claim over all execution paths, whose value derives from deduction over the source, not from any observation. A witness record with no boundary crossing has nothing to preserve; a formation proof with no execution is already complete. The two artifacts answer different questions and are consumed differently by a machine reader.

Accommodation would require the witness definition to read, in effect, “preserves evidence that conditions held or must hold.” That single disjunction merges an existential operation and a universal one under one vocabulary term, making them indistinguishable to exactly the consumers GLM serves. This is the semantic collapse the standard is built to prevent, and it is the failure mode already identified, in writing, by both parties to this proposal.

### **3.6 Result.**

No existing functional type accommodates the formation proof object without redefinition, and the nearest type fails precisely at the modality the object is built around. The documented-rationale-plus-schema-increment path defined by the standard is therefore the correct mechanism, and the remainder of this document supplies that rationale.

## **4. Dimensions of distinction**

### **4.0 Frame.**

The proposing parties identified three dimensions on which a formation proof object differs from every existing functional type. All three were named in the prior written exchange; this section orders them by argumentative weight rather than by the order in which they were first raised: the accommodation test produced the result that the timing dimension nests inside the witness window, and the ordering follows that result.

**4.1 Function: universal quantification over execution paths rather than observation of a specific boundary crossing.**

This is the load-bearing dimension, the one on which the accommodation test failed. The two operations differ in quantifier, in evidentiary source, and in failure mode, per Section 3.5; no reading of the witness definition spans both without the disjunction that produces collapse.

**4.2 Timing: pre-execution rather than before-or-at-bind.**

This dimension is real but corroborating. On the bind-time axis pre-execution nests inside before-or-at-bind, which is why it cannot discriminate alone. What is distinct is the reference frame, not the clock position: witness timing is event-relative, taking its meaning from a boundary crossing, while formation timing is source-anchored, taking its meaning from an artifact, per Section 2.3. The proposed entry therefore declares the existing vocabulary position pre-bind, qualified source-anchored, rather than requesting a new position value. The qualification follows the precedent set by substrate’s “(continuous)”.

**4.3 Execution capability: false. A proof object rather than an actor.**

A formation proof cannot execute, authorize, halt, or intervene; it has no runtime surface at all. This dimension does shared-posture work: the reference implementation of the closure type declares the same value, and the two declarations together demonstrate the pattern of governance layers placing themselves at a machine-readable distance from the execution layer. `machine_readable_status.execution_capability` is false in any conforming formation manifest.

**4.4 The combination.**

Independent verifiability is not a fourth dimension. GLM requires of every declared layer that its artifacts be externally verifiable; a conforming formation manifest inherits that requirement rather than introducing it. Given that shared baseline, no single dimension justifies a vocabulary entry. A pre-bind artifact is not new. A non-executing layer is not new either. A universal claim on its own would be a property of a verification tool, not of a governance layer. The position that justifies the entry is the conjunction: a sealed, independently verifiable proof object, source-anchored, establishing that declared conditions must hold across all execution paths before any execution exists, with no execution capability. The prior exchange anticipated that this combination might justify a distinct `layer_type`; Sections 2 and 3 are the disciplined mapping that confirms it.

**5. Proposed vocabulary entry**

The proposed addition to the `layer_type` vocabulary table, in the register of the existing rows:

| layer_type | Timing surface             | Primary operational function                                                                                                                                                                |
|------------|----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| formation  | Pre-bind (source-anchored) | Establishes that declared conditions must hold across all execution paths, universally quantified, before any specific execution occurs. It is a proof object, not a runtime control layer. |

A conforming formation manifest declares its timing axis as follows:

```
"timing_axis": {  
  "position": "pre-bind",  
  "description": "Operates before any execution bind exists. The  
    temporal anchor is the source artifact, not a bind event: the  
    proof is formed when the system is formed and holds for every  
    execution path derivable from that source. It exists prior to  
    and independent of any specific execution instantiation. Does  
    not operate at bind or post-bind, and does not attach to any  
    individual execution."  
}
```

Its definitions block carries the term defined in Section 2.1, verbatim:

```
"formation_proof_object": "A sealed, independently verifiable  
artifact establishing that declared conditions must hold across  
every execution path derivable from a specific source artifact,  
formed before any execution exists. It establishes that conditions  
must hold; it does not record that conditions held during any  
specific execution."
```

The “source-anchored” qualification of the pre-bind position is a declared timing qualifier, added to the schema in the same increment (Section 10), not explanatory prose. It is defined for machine readers as follows, in the register of the existing “continuous” qualifier on the substrate row:

```
"source-anchored": "A timing qualifier on a pre-bind position. The  
temporal reference frame is the source artifact, not a bind event:  
the qualified layer's output is anchored to a source and holds for  
every bind of that source, rather than being taken relative to one  
particular bind. Qualifies a position; does not introduce a new  
position on the timing axis."
```

A reader resolves “Pre-bind (source-anchored)” in the table cell against this definition without the surrounding text.

## 6. Claims boundary

### 6.0 Frame.

Field semantics follow GLM v2.0; the non-claim and forbidden-inference distinction follows Section 2.4. The entries below are type-level: what any conforming formation manifest must declare. Vendor-specific capability declarations (operational\_scope.does) belong to the individual manifest; the entries below are the taxonomy-protective floor.

#### 6.1 authoritative\_layer\_claim (one sentence, per the spec):

*Formation establishes, before any execution exists, that declared conditions must hold across every execution path derivable from a specific source artifact, sealed as an independently verifiable proof object.*

#### 6.2 operational\_scope.does\_not:

```

0  Resolve admissibility at bind -- that is the boundary layer
    function
1  Preserve evidence of governed-state conditions surrounding a
    boundary crossing -- that is the witness layer function
2  Record behavioral history -- that is the substrate layer function
3  Stabilize responsibility-bearing closure states -- that is the
    closure layer function
4  Test artifacts of other layers for independent reviewability --
    that is the reviewability layer function
5  Enforce, authorize, halt, or intervene in execution
6  Attest the behavior of any specific execution
7  Certify that the system is safe or compliant with any regulation
8  Validate the adequacy of the declared conditions themselves --
    the proof establishes consequence from declarations, not the
    merit of the declarations

```

### 6.3 `claims_boundary.explicit_non_claims`:

```

0  Does not determine admissibility
1  Does not attest any specific execution
2  Does not certify regulatory compliance
3  Does not certify overall system safety
4  Does not validate the adequacy of declared conditions
5  Does not hold or transfer execution authority

```

### 6.4 `consumer_boundary_constraint`:

*Consumers must not interpret formation outputs as execution authorization, runtime approval, execution-conformance attestation, certification of regulatory compliance, or certification of system safety. A proof that declared conditions must hold across all execution paths must not be re-interpreted as evidence that any particular execution occurred or conformed.*

### 6.5 `claims_boundary.forbidden_interpretations`:

```

0  execution_authorization
1  runtime_approval
2  authority_transfer
3  execution_conformance_attestation
4  occurrence_attestation
5  compliance_certification
6  safety_certification

```

## 7. Semantic collapse analysis

### 7.0 Frame.

Collapse, as defined in the prior exchange: two structurally different functions sharing a vocabulary that makes them appear equivalent to a machine reader. This section traces what a machine reader concretely gets wrong under a witness labeling, in both directions, and why field-level protections cannot repair a type-level error.

### 7.1 Over-attribution.

A consumer dispatching on `layer_type`: witness applies witness semantics: an observation occurred, a boundary context existed, conditions held at a specific moment. All three are false of a formation proof,

which establishes consequence by deduction over the source with no observation and no boundary crossing, per Section 3.5. The consumer has concluded that something happened. `occurrence_attestation` is the token that forbids this inference, but under the witness label the consumer never reads it as anomalous, because `occurrence` is what the type is for.

## **7.2 Under-attribution.**

The same labeling degrades the claim in the opposite direction. The universal guarantee, must hold across every path, is read as an existential record, held at one moment. A downstream boundary layer that could rely on the universal claim as an admissibility condition has no machine-readable way to know the claim is universal; the strongest property of the artifact is silently discarded.

## **7.3 The laundering chain.**

The two directions compose into the dangerous path: a formation proof labeled witness is read as evidence that conditions held at bind; held-at-bind evidence is treated as admissibility input for a specific execution; admissibility input shades into runtime approval. Each step is locally plausible; the conjunction converts a pre-execution deduction into an execution permission. The tokens `execution_conformance_attestation` and `runtime_approval` cut this chain at its two final links, and they can only do so if the first link, the `occurrence` misreading, is blocked at the type level.

## **7.4 Why field-level protection is insufficient.**

GLM's protection mechanism, `forbidden_interpretations`, constrains how consumers read the outputs of a correctly typed layer. It cannot repair a wrong type, because `layer_type` is the first discriminator a machine reader dispatches on; every field is interpreted inside the frame the type establishes. A witness-typed manifest whose tokens deny `occurrence` is not a guarded declaration but a self-contradictory one, and a machine reader resolves the contradiction in favor of the type. The protection the standard provides therefore presupposes correct typing; it cannot substitute for it.

## **7.5 The controlling test.**

The proposed entry passes the standard's own test: "Interoperability must increase reviewability without silently transferring authority across layers." Reviewability increases: a reviewer can distinguish a universal proof from an existential record by machine, with no bilateral inquiry. No authority moves: `execution_capability` is false, and the retained tokens `execution_authorization`, `runtime_approval`, and `authority_transfer` declare the distance machine-readably. The collapse this section traces is, precisely, a silent authority transfer toward execution, `occurrence` misread as permission, and the entry exists to make it structurally detectable rather than silent.

# **8. Composition posture**

## **8.0 Frame.**

GLM creates interoperability through independently readable boundary declarations, not through integration agreements. This section declares how a formation layer composes with each functional type and what the composition does not imply. Composability declarations are unilateral under the spec; nothing here binds any existing manifest.

## **8.1 composable\_with\_types:**

substrate, witness, boundary, closure, reviewability

All five. The mapping test separated formation from each type by function; separation by function is what makes composition coherent rather than redundant. Per type:

- **substrate.** Parallel on the shared pre-bind surface, no artifact exchange required. A substrate signal may inform what conditions a source declares; the proof then ranges over the declared conditions, per `does_not` entry 8, not over the signal.
- **witness.** Complementary modalities around the same boundary: formation establishes that conditions must hold on every path; a witness preserves evidence of the governed-state conditions surrounding the actual crossing. A stack carrying both holds the universal claim and the existential record, and a machine reader can tell which is which. This composition is only coherent because the types are distinct; under collapse it degenerates into two witnesses, per Section 7.2.
- **boundary.** The principal downstream consumer. A formation proof is one admissibility condition among those a boundary layer resolves at bind, per Section 3.2. The constraint travels with the artifact: consuming the proof as a condition is composition; treating it as the admissibility resolution itself is the forbidden inference `execution_authorization`.
- **closure.** The mirror relation, per Section 3.3. A formation-side conformance record is one of the conditions present at closure; the closure layer stabilizes the responsibility state that includes it. The two types occupy opposite ends of the timing axis and share the structural property the prior exchange named: both preserve reconstructability before ambiguity emerges.
- **reviewability.** Formation artifacts are subjects of reviewability testing, per Section 3.4. The composition is first-order to second-order: the proof object is built to satisfy independent-review conditions, and a reviewability layer can confirm that property without access to the originating environment.

## 8.2 composable\_with\_manifests.

Empty at type level. Specific manifest URLs are vendor declarations made when verified, per the spec's `composition_note` pattern. The type proposal declares none.

## 8.3 What composition does not imply.

Composability declarations describe structural fit, not semantic inheritance: a composed stack does not acquire any layer's claims transitively. In particular, a stack containing a formation layer does not thereby attest occurrence, conformance, or compliance of anything executed downstream; each layer's `forbidden_interpretations` survive composition intact. The spec's `vendor_framework_constructs` block states the same principle for constructs; this entry states it for the stack.

# 9. Reference implementation evidence

## 9.0 Frame.

This section is evidence of occupancy, not part of the taxonomy argument: Sections 2 through 7 stand without it. It documents that the proposed type has a shipped implementation whose claims are bounded by released, installable state (nous-lang 5.35.0 on PyPI at the time of writing), and that the type's claims boundary is enforced in that implementation rather than only asserted in this document. All capability statements below are vendor-specific and belong to the implementation, not to the type.



### **9.1 The proof object as shipped.**

A build produces a signed manifest sealing formation-time proofs over the source: a cost ceiling proved across all execution paths, ordering laws, and cardinality bounds, each established by an SMT solver before any execution exists. The dossier carries its own offline verifier; a third party verifies the seal and the bound claims with a cryptography library and a standard Python runtime only, with no solver and no vendor software in the verification path. Where the manifest binds a coverage claim, it carries an algebraic certificate the verifier checks by rational arithmetic, so even the coverage proof is solver-free on the consumer side.

### **9.2 The checkable fragment.**

The solver-free certificate covers the disjunctive linear-arithmetic fragment. A single comparison, a flat disjunction of comparisons, and multi-variable linear combinations within a comparison are checked directly; boolean blocking structure (conjunction, disjunction, negation) is checked by expanding the negation of the claim to disjunctive normal form and refuting every disjunct with non-negative rational multipliers. The verifier does not trust the producer's enumeration: it re-derives the disjunct set from the signed source and demands a one-to-one match between derived disjuncts and certificates, so a certificate that omitted a disjunct would fail rather than pass. Nonlinear terms, a variable multiplied by a variable, are refused outright: within that fragment no cheap standard-library certificate exists, and the implementation declines to sign what its consumers could not independently check rather than widening the claim. The refused edge is the boundary of Section 6 in operational form; the checkable interior is wider than a single comparison, and every claim inside it is verified by rational arithmetic with no solver on the consumer side.

### **9.3 Three obligations, one primitive.**

A change to the source requires a new proof, per Section 2.3; the implementation binds each new proof to its predecessor by signed digest, and a full-mode dossier carries each prior link's source under that link's signed source hash. The offline verifier walks the chain to genesis under fail-closed conditions and discharges three distinct obligations through the same re-derivable certificate mechanism, each re-derived from signed inputs rather than attested by signature: that the coverage claim holds at each link; that the declared threshold region did not shrink across a re-binding hop (region containment, expressed as the unsatisfiability of predecessor-region and not-successor-region over the reals); and that the actual blocking net did not shrink across the chain (net containment, re-derived from the carried sources, never from a bundle). One proof engine carries all three. The boundary travels verbatim with the artifact: each obligation proves that a declared region or net did not shrink, not that any version is safer, not execution conformance; the declared policies are monitors, not guards.

### **9.4 The execution boundary.**

The implementation also emits per-run conformance certificates that check an execution trace against the formation envelope. These are deliberately separate artifacts, bound to the formation proof by digest reference, never folded into it: execution evidence composes with the formation proof, per Section 8, instead of widening it. The certificate itself carries its own limit, proving that the trace conforms, not that the trace faithfully records reality. The vendor's own non-claims thus reproduce does\_not entry 6 in shipped behavior.

### **9.5 External consumption.**

Formation-time evidence from this implementation is structured for downstream audit consumption: the dossier format maps clause-by-clause onto documentation obligations that auditors evaluate, that crosswalk is maintained as working reference material, and audit-side practitioners have adopted the evidence categories into audit methodology. The category therefore has a consumer side that is neither the vendor nor the standard's maintainer. No more is claimed: the regulatory mapping is evidence of the consumer side, not justification for the type.

## **10. Schema impact**

### **10.0 What is requested.**

Two additions to the schema, in one increment. First, the value `formation` is added to the controlled `layer_type` vocabulary, with the table row given in Section 5. Second, `source-anchored` is added as a declared timing qualifier, with the definition given in Section 5, so the qualification of the pre-bind position is machine-readable rather than carried in prose. Both follow the path the standard defines for extension, a documented rationale plus a schema version increment; this document is that rationale. Nothing else is requested.

### **10.1 Increment class.**

Under the standard's stated policies, breaking changes require a major version increment and non-breaking additions a minor one, backward compatible within the major version. Neither addition is breaking. Adding a vocabulary value invalidates no existing manifest, changes no field, and alters no semantics of any existing type. Adding a timing qualifier likewise invalidates no existing manifest: the existing "continuous" qualifier on the substrate row establishes that positions carry qualifiers already, and naming a second one removes no qualifier and changes no position. Both additions are therefore non-breaking, and the increment is a minor one: `schema_version` 1.0 to 1.1.

### **10.2 What does not change.**

The five functional types and `cross_cutting` are untouched. No new field is introduced. No new timing position is introduced; per Section 4.2, `formation` declares the existing position pre-bind, and `source-anchored` qualifies that position rather than adding one to the axis. No existing manifest is required to change: the `schema_version` / `manifest_version` separation introduced in the GLM v2.0 revision means existing manifests continue to declare schema 1.0 validly, and migrate on their own clock.

### **10.3 Compatibility window.**

The increment lands inside compatibility windows already declared in the field. The reference implementation of the closure type declares `minimum_supported` 1.0 and `forward_compatible_until` 1.x, with non-breaking additions as minor increments; a 1.1 schema is inside that declared window by the declaring manifest's own terms. No coordination is required, which is the interoperability doctrine applied to the schema change itself.

### **10.4 Effects available, none required.**

Existing layers may unilaterally add `formation` to `composable_with_types`, per the spec's unilateral-declaration model; none is obliged to. Tooling that compares manifests gains one additional `layer_type` value and one additional timing qualifier to discriminate on, which is the entire machine-facing effect of the proposal.

### **10.5 Status of this exercise.**

The vocabulary was designed as extensible through documented rationale and schema revision; this proposal exercises that mechanism as specified, and is, to the proposing parties' knowledge, its first external exercise. No precedence, authority, or privileged status is claimed for the proposing implementation, matching the standard's own vendor-note posture toward its initial proposing implementation.

---

End of document.